

H2O Online Security Assessment & Remediation Guide

Comprehensive end-to-end security review of the H2O Online platform — backend API, admin portal, mobile application, payment flows, and AWS deployment. Includes live test results, detailed findings (authentication, authorization, API, mobile, payments, network), OWASP Top 10 mapping, required fixes, full remediation catalog (80+ options), verification checklist, and PCI-DSS notes.

Assessment covered:

- Authentication, RBAC, session/token management, API security
- Input validation, injection, business logic & payment bypass testing
- Mobile & admin portal security, network/TLS, data security
- Dependency scanning, OWASP Top 10, URL/page access control
- Live probes on production API (July 2026)

5

Critical Issues

10

High Severity

8

Medium

80+

Remediation Options

11

Document Sections

Contents

- | | |
|--------------------------------------|----------------------------------|
| 1. Executive Summary | 7. Required Fixes (Priority 0–2) |
| 2. Scope & Methodology | 8. Possible Fixes Catalog |
| 3. Architecture & Trust Boundaries | 9. Verification Checklist |
| 4. Live Test Results | 10. PCI-DSS & Privacy |
| 5. Detailed Findings (12 categories) | 11. Appendix: Key Files |
| 6. URL & Page Access Control | |

Version: 1.0**Assessment date:** July 2026**Production API:** <http://13.62.57.255:5000>**Production Admin:** <http://13.62.57.255:3000>**Generated:** 6 July 2026**CONFIDENTIAL — INTERNAL USE ONLY**

H2O Online — Security Assessment & Remediation Guide

Version: 1.0

Assessment date: July 2026

Scope: Backend API, Admin SPA, Mobile app (Expo), payment flows, deployment

Production targets tested: `http://13.62.57.255:5000` (API), `http://13.62.57.255:3000` (Admin)

This document records the end-to-end security and vulnerability review performed on the H2O Online application. It explains what was tested, what was found, **fixes that are required before production use with real payments**, and a **catalog of all possible remediations** you can implement over time.

Table of Contents

1. Executive Summary

2. Assessment Scope & Methodology

3. Architecture & Trust Boundaries

4. Live Test Results

5. Detailed Findings by Category

- 5.1 Authentication

- 5.2 Authorization & RBAC

- 5.3 Session & Token Management

- 5.4 API Security

- 5.5 Input Validation & Injection

- 5.6 Business Logic & Payment Security

- 5.7 Mobile Application Security

- 5.8 Admin Portal Security

- 5.9 Network & Transport Security

- 5.10 Data Security

- 5.11 Dependency Vulnerabilities

- 5.12 OWASP Top 10 Mapping

1. URL & Page Access Control

2. Required Fixes (Must Do Before Production)

3. Possible Fixes Catalog (Full Remediation Options)

4. Verification & Re-Test Checklist

5. Compliance Notes (PCI-DSS & Privacy)

6. Appendix: Key Files Reference

1. Executive Summary

Overall posture

The H2O platform has a **reasonable baseline** for a startup MVP: JWT authentication, bcrypt password hashing, Mongoose ODM, Razorpay signature verification, and admin/customer API separation in many areas.

However, **several critical business-logic and configuration flaws** were confirmed on the live production API. These allow:

- Unlimited wallet self-funding (payment bypass)
- Default master admin login on production
- Client-controlled order pricing
- Non-Razorpay orders marked as “paid”
- Sensitive tokens transmitted over **HTTP**

Risk summary

Severity	Count	Examples
Critical	5	Wallet self-credit, default admin creds, price manipulation, payment bypass, HTTP in production
High	10	No rate limiting, no MFA, JWT in insecure storage, secrets in <code>eas.json</code> , Razorpay amount not verified
Medium	8	Sub-admin over-permission, public tax settings, ReDoS in search, no webhooks
Low / Informational	6	No Helmet, mock OTP UI, client-only logout

Recommendation

Do not go live with real customer payments until all **Section 7 (Required Fixes)** items are completed and re-tested using **Section 9**.

2. Assessment Scope & Methodology

In scope

Area	What was reviewed
Authentication	Login, register, admin login, password handling, MFA/OTP screens
Authorization	RBAC (customer, supplier, rider, admin, sub-admin), API route guards
Session management	JWT expiry, storage, logout, refresh
API security	Unauthenticated access, IDOR samples, wallet/payment endpoints
Input validation	Order body, search filters, Mongoose usage
Business logic	Wallet credit, order pricing, Razorpay verify flow
Mobile	Token storage, route guards, hardcoded secrets, HTTP API URL
Admin SPA	Route guards, token storage, backend enforcement
Network	HTTP vs HTTPS, CORS configuration
Dependencies	npm audit on backend, admin, mobile

Methods used

- 1. **Static code review** — middleware, routes, services, models, client API clients
- 2. **Live API probing** — production backend with test credentials (read-only where possible; wallet credit test documented)
- 3. **Dependency scanning** — npm audit
- 4. **Architecture review** — auth flows, payment flows, role boundaries

Out of scope (not performed in this pass)

- Full manual penetration test (Burp/ZAP on all endpoints)
- Mobile APK reverse engineering (MobSF, jadx)
- MITM lab testing with proxy on physical devices
- Load/stress testing
- Formal PCI-DSS audit
- Social engineering / phishing tests

These are listed as **possible fixes / follow-up activities** in Section 8.

3. Architecture & Trust Boundaries



Trust assumptions (current)

Component	Trusts	Risk if violated
Backend	Client sends correct price , total , paymentMethod	Price/payment bypass
Backend	Any authenticated user on /api/wallet/credit	Unlimited wallet balance
Mobile	AsyncStorage is private	Token theft on rooted device
Admin	localStorage + no XSS	Admin token theft
Network	HTTP is acceptable	MITM, token interception

What should be trusted (target state)

- **Only the backend** determines prices, payment status, and wallet balances.
- **Only Razorpay webhooks + verified client callback** confirm online payment.

- **HTTPS everywhere** for API and admin.
- **Short-lived tokens** with refresh or server-side revocation.

4. Live Test Results

Tests run against `http://13.62.57.255:5000` in July 2026.

4.1 Unauthenticated access (expected: 401)

Endpoint	Method	Result	Verdict
<code>/api/health</code>	GET	200	OK — intentionally public
<code>/api/orders</code>	GET	401 No token	OK
<code>/api/admin/orders</code>	GET	401 No token	OK
<code>/api/admin/financials</code>	GET	401 No token	OK
<code>/api/wallet/credit</code>	POST	401 No token	OK
<code>/api/settings/tax</code>	GET	200 (full tax config)	Over-exposed
<code>/api/settings/payment</code>	GET	200 (Razorpay key id)	Acceptable for checkout

4.2 Authenticated authorization (customer token)

Test	Result	Verdict
POST <code>/api/wallet/credit { amount: 5000 }</code>	200 — balance increased	CRITICAL FAIL
GET <code>/api/supplier/orders/incoming</code>	403 Supplier profile required	OK
GET <code>/api/admin/orders</code>	403 Access denied: admin portal only	OK
GET <code>/api/orders/{randomId}</code>	404 Order not found	OK (no IDOR in sample)

4.3 Admin authentication

Test	Result	Verdict
POST <code>/api/admin/auth/login</code> with H2O admin / <code>admin@H2O</code>	200 + JWT	CRITICAL FAIL

4.4 Payment bypass (partial)

Test	Result	Verdict
POST <code>/api/orders</code> with <code>paymentMethod: "card"</code> , <code>price: 1</code>	400 (serviceability)	Blocked by address check, not by payment logic

Note: Code review confirms that if serviceability passes, `paymentMethod: "card"` sets `paymentStatus: "paid"` without gateway verification.

4.5 Test side effect

A wallet credit probe added **₹5,000** to the test customer account (`rohit@gmail.com`). Reverse via **Admin** → **Wallet management** if unintended.

5. Detailed Findings by Category

5.1 Authentication

ID	Finding	Severity	Details
AUTH-01	Default master admin credentials	Critical	<code>backend/middleware/adminAuth.js</code> defaults to <code>H2O admin / admin@H2O</code> if env vars missing. Confirmed working on production.
AUTH-02	Weak password policy	High	Minimum 6 characters only (<code>backend/models/User.js</code>). No complexity, history, or breach check.
AUTH-03	No MFA / 2FA	High	<code>mobile/src/screens/LoginOTPScreen.js</code> is mock UI — no backend OTP verification.
AUTH-04	No password reset	High	<code>ForgotPasswordScreen.js</code> shows "Coming soon"; no reset API.
AUTH-05	No rate limiting on login	High	Unlimited login attempts on <code>/api/auth/login</code> and <code>/api/admin/auth/login</code> .
AUTH-06	Generic login errors	Good	Returns "Invalid email or password" — no user enumeration via message.
AUTH-07	bcrypt password hashing	Good	10 rounds, <code>select: false</code> on password field.

Files: `backend/routes/auth.js`, `backend/routes/adminAuth.js`, `backend/models/User.js`

5.2 Authorization & RBAC

ID	Finding	Severity	Details
AUTHZ-01	Wallet self-credit for any user	Critical	POST /api/wallet/credit adds arbitrary balance. Used by mobile wallet top-up flow.
AUTHZ-02	No centralized requireRole() middleware	Medium	Most routes use auth then per-handler checks; inconsistent.
AUTHZ-03	Sub-admin broad write access	High	Can adjust wallets, change tax settings, verify suppliers — only 4 actions restricted.
AUTHZ-04	Admin JWT vs mobile JWT same secret	Medium	Admin user token could theoretically be used on mobile API (role not blocked in auth.js).
AUTHZ-05	Supplier routes check profile	Good	Returns 403 if no supplier profile.
AUTHZ-06	Admin routes use adminAuth	Good	Non-admin roles get 403 on admin API.
AUTHZ-07	Financials restricted to master/admin	Good	requireCanSeeFinancials on /api/admin/financials.
AUTHZ-08	Delivery partner list visible to all auth users	Low	GET /api/delivery-partners/ exposes names/phones to any logged-in user.

Files: backend/routes/wallet.js , backend/middleware/auth.js , backend/middleware/adminAuth.js , backend/routes/admin.js

5.3 Session & Token Management

ID	Finding	Severity	Details
SESS-01	JWT expiry 7 days, no refresh	High	Long-lived stolen token window.
SESS-02	No server-side logout / revocation	High	Logout clears client storage only; token valid until expiry.
SESS-03	No token denylist / rotation	Medium	Cannot invalidate compromised tokens.
SESS-04	Mobile token in AsyncStorage	High	Key: h20_auth_token — not encrypted.
SESS-05	Admin token in localStorage	High	Key: h20_admin_token — XSS-vulnerable.

Files: mobile/src/context/AuthContext.js , admin/src/context/AuthContext.jsx , admin/src/api/client.js

5.4 API Security

ID	Finding	Severity	Details
API-01	No rate limiting	High	No express-rate-limit or similar.
API-02	No Helmet security headers	Medium	Missing X-Content-Type-Options , X-Frame-Options , etc.
API-03	CORS allows all origins in development	Medium	backend/server.js — ensure production uses ALLOWED_ORIGINS .
API-04	Body size limit 2MB	Good	Limits oversized payloads.
API-05	Bearer token auth (not cookies)	Good	CSRF low priority for current design.
API-06	Public tax settings endpoint	Medium	GET /api/settings/tax exposes commission/GST without auth.

Files: backend/server.js , backend/routes/settings.js

5.5 Input Validation & Injection

ID	Finding	Severity	Details
INJ-01	Mongoose ODM used throughout	Good	Reduces classic NoSQL injection.
INJ-02	ReDoS via unescaped RegExp search	Medium	new RegExp(search, "i") in admin/products/supplier search.
INJ-03	No validation library (zod/joi)	Medium	Ad-hoc checks per route; easy to miss fields.
INJ-04	Mass assignment on user profile	Low	Email change via PUT /api/users/me without re-auth.
INJ-05	XSS in mobile WebView (Razorpay)	Low	Use trusted Razorpay checkout URLs only; validate WebView source.

Files: backend/routes/admin.js , backend/routes/products.js , backend/routes/users.js

5.6 Business Logic & Payment Security

ID	Finding	Severity	Details
PAY-01	Client-controlled item prices	Critical	Order uses i.price from request; not verified against Product.price in DB.
PAY-02	Fake “paid” orders via paymentMethod: "card"	Critical	Default payment method marks order paid without gateway.
PAY-03	Wallet self-credit → free orders	Critical	Credit wallet, pay with wallet — no real money.
PAY-04	Razorpay signature verified	Good	HMAC-SHA256 in backend/services/razorpay.js .
PAY-05	Duplicate payment ID check	Good	In-memory check before order create.
PAY-06	Razorpay amount not verified	High	Fetches payment amount not compared to order total.
PAY-07	No Razorpay order linkage at verify	High	razorpay_order_id from create-order not stored/checked.
PAY-08	No unique DB index on razorpayPaymentId	Medium	Race could create duplicate orders.
PAY-09	No Razorpay webhook handler	Medium	Relies on client callback only.

ID	Finding	Severity	Details
PAY-10	Signature compare not timing-safe	Low	Use <code>crypto.timingSafeEqual</code> .
PAY-11	Payment fetch failure still creates order	Medium	Falls back to <code>{ paidAt: new Date() }</code> if Razorpay fetch fails.
PAY-12	Direct Razorpay blocked on POST /orders	Good	Must use verify-payment endpoint.
PAY-13	Billing totals cross-checked	Good	<code>validateOrderBilling</code> checks subtotal/tax/total consistency.

Files: `backend/routes/payments.js`, `backend/services/customerOrderService.js`, `backend/routes/wallet.js`, `backend/routes/orders.js`

5.7 Mobile Application Security

ID	Finding	Severity	Details
MOB-01	No global route guard	Medium	Expo routes in <code>_layout.tsx</code> — screens can open without login; API enforces data.
MOB-02	JWT in AsyncStorage	High	Not using <code>expo-secure-store</code> .
MOB-03	Hardcoded secrets in <code>eas.json</code>	High	Google Maps key, Razorpay test key, HTTP API URL in build config.
MOB-04	Production API over HTTP	Critical	Tokens and payment data visible on network.
MOB-05	No certificate pinning	Medium	Standard TLS trust store only (when HTTPS enabled).
MOB-06	No root/jailbreak detection	Low	Optional for high-security apps.
MOB-07	Mock OTP screen	Informational	Could mislead users into thinking MFA exists.

Files: `mobile/eas.json`, `mobile/app/_layout.tsx`, `mobile/src/context/AuthContext.js`, `mobile/config/apiUrl.json`

5.8 Admin Portal Security

ID	Finding	Severity	Details
ADM-01	SPA route guards via React	Good	<code>PrivateRoute</code> redirects unauthenticated users to <code>/login</code> .
ADM-02	Financials/Admin users UI-only guard	Medium	Backend must enforce (partially done).
ADM-03	Static files served without auth	Expected	API is the real security boundary.
ADM-04	Token in localStorage	High	Vulnerable if XSS introduced in admin bundle.

Files: `admin/src/App.jsx`, `admin/src/context/AuthContext.jsx`

5.9 Network & Transport Security

ID	Finding	Severity	Details
NET-01	HTTP API in production builds	Critical	No TLS on <code>13.62.57.255:5000</code> .
NET-02	HTTP admin portal	High	Same MITM risk for admin JWT.
NET-03	CORS credentials enabled	Low	OK with Bearer tokens; review if cookies added later.

5.10 Data Security

ID	Finding	Severity	Details
DATA-01	Passwords hashed at rest	Good	bcrypt in MongoDB.
DATA-02	JWT secret in env	Good	If strong and not default.
DATA-03	Default JWT secret fallback	High	"h20-secret" if <code>JWT_SECRET</code> not set.
DATA-04	Razorpay secret server-only	Good	Not in mobile/admin clients.
DATA-05	PII in orders/addresses	Informational	Ensure MongoDB Atlas encryption at rest (Atlas default).

5.11 Dependency Vulnerabilities

`npm audit` summary (July 2026):

Package	Critical	High	Moderate	Low
backend	0	3	2	0
admin	0	1	4	1
mobile	0	1	15	0

Notable backend highs: `minimatch`, `path-to-regexp`, `picomatch` (ReDoS / glob issues — mostly dev/transitive).

Action: Run `npm audit fix` where safe; upgrade major deps in a controlled branch.

5.12 OWASP Top 10 Mapping

OWASP 2021	H2O status	Primary findings
A01 Broken Access Control	Fail	Wallet credit, sub-admin scope, price control
A02 Cryptographic Failures	Fail	HTTP in production, insecure token storage
A03 Injection	Partial	Mongoose OK; ReDoS in search
A04 Insecure Design	Fail	Client-trusted prices and payment status
A05 Security Misconfiguration	Fail	Default admin creds, secrets in eas.json
A06 Vulnerable Components	Partial	npm audit findings
A07 Auth Failures	Fail	No MFA, no rate limit, weak passwords
A08 Software/Data Integrity	Partial	No webhook verification for Razorpay
A09 Logging & Monitoring	Partial	Basic console logs; no SIEM/alerting
A10 SSRF	Low risk	Limited outbound URL fetching

6. URL & Page Access Control

Understanding what users can **see in the browser/app** vs what the **API allows** is essential.

6.1 Admin portal

URL path	Without login	With login (sub-admin)	With login (master/admin)
/login	Visible	Redirects to dashboard	Redirects to dashboard
/, /orders, /suppliers	Redirect to /login	Visible	Visible
/financials	Redirect to /login	UI redirect to /	Visible
/admin-users	Redirect to /login	UI redirect to /	Visible

Important: The admin app is a SPA. Static HTML/JS loads for any URL; **React Router** redirects unauthenticated users. The **real enforcement is the API** — always verify with curl/Postman, not only by clicking URLs.

6.2 Mobile app (Expo Router)

Route example	Without login	With customer login	With supplier login
/login	Visible	Can navigate	Can navigate
/dashboard	May open; API calls fail or empty	Works	Wrong role UX
/supplier-dashboard	May open	API 403	Works
/payment, /checkout	May open	Works if cart filled	N/A

Gap: No global auth wrapper in `mobile/app/_layout.tsx`. Individual screens check `isAuthenticated` inconsistently. **Backend must remain the authority.**

6.3 How to verify page access (QA steps)

Admin — unauthenticated

1. Open incognito window.
2. Visit `http://13.62.57.255:3000/orders`.
3. **Expected:** Redirect to `/login`; API calls return 401.

Admin — sub-admin escalation

1. Log in as sub-admin.
2. Open DevTools → Network.
3. Call `GET /api/admin/financials` manually.
4. **Expected:** 403 Permission denied.

Mobile — deep link

1. Log out.
2. Navigate to `/supplier-wallet` (or deep link).
3. **Expected:** UI may render; wallet API returns 401 or 403.

API — always test separately from UI

```
# No token – expect 401
curl -s http://13.62.57.255:5000/api/orders

# Customer token – expect 403
curl -s http://13.62.57.255:5000/api/admin/orders \
-H "Authorization: Bearer <customer_jwt>"
```

7. Required Fixes (Must Do Before Production)

These fixes are **mandatory** before accepting real payments or public launch.

Priority 0 — Immediate (same day)

#	Fix	Why	Where to change
R0-1	Remove or restrict <code>POST /api/wallet/credit</code>	Confirmed exploit: any user adds unlimited balance	<code>backend/routes/wallet.js</code> — remove route or restrict to admin-only with audit log
R0-2	Change master admin credentials	Default <code>H2O admin / admin@H2O</code> works on production	<code>backend/.env</code> on server: <code>MASTER_ADMIN_EMAIL</code> , <code>MASTER_ADMIN_PASSWORD</code>
R0-3	Set strong <code>JWT_SECRET</code>	Prevent token forgery	<code>backend/.env</code> : <code>JWT_SECRET=<32+ random bytes></code>
R0-4	Reverse test wallet credit	₹5,000 added during security test	Admin → Wallet management → adjust customer wallet

Priority 1 — Before real payments (1–2 weeks)

#	Fix	Why	Where to change
R1-1	Server-side product pricing	Load <code>Product.price</code> from DB; reject client price mismatch	<code>backend/services/customerOrderService.js</code>
R1-2	Block fake paid orders	Only <code>cod</code> → pending; <code>razorpay</code> → via verify only; <code>wallet</code> → debit balance	<code>customerOrderService.js</code> , <code>orders.js</code>
R1-3	Verify Razorpay payment amount	Compare fetched payment amount (paise) to order total	<code>backend/routes/payments.js</code>
R1-4	Link Razorpay order ID	Store <code>razorpay_order_id</code> at create; verify same ID at payment	<code>payments.js</code> , <code>Order</code> model
R1-5	Enable HTTPS	Stop MITM on JWT and payment data	Nginx/Caddy + Let's Encrypt on EC2; update mobile/admin URLs
R1-6	Remove secrets from <code>eas.js</code> on	Keys in git history	EAS Secrets; env at build time only
R1-7	Add rate limiting on auth	Stop brute force	<code>express-rate-limit</code> on <code>/api/auth/login</code> , <code>register</code> , <code>admin login</code>

Priority 2 — Hardening (2–4 weeks)

#	Fix	Why
R2-1	Move mobile tokens to expo-secure-store	Protect JWT on device
R2-2	Add Razorpay webhooks	Reconcile payments server-side
R2-3	Unique index on <code>razorpayPaymentId</code>	Prevent duplicate orders
R2-4	Tighten sub-admin RBAC	Restrict wallet adjust, tax settings
R2-5	Stronger password policy	Min 8–12 chars, complexity rules
R2-6	Helmet + security headers on API	Standard HTTP hardening
R2-7	Escape user input in RegExp search	Prevent ReDoS

8. Possible Fixes Catalog (Full Remediation Options)

This section lists **all possible security improvements** — including items beyond Section 7. Use it as a roadmap; not everything is required for MVP, but each improves posture.

8.1 Authentication & identity

Fix	Description	Effort	Impact
MFA / TOTP	Authenticator app or SMS OTP after password	High	High
Email OTP login	Implement backend for <code>LoginOTPScreen.js</code>	Medium	High
Password reset flow	Forgot password → email link → reset API	Medium	High
Account lockout	Lock after N failed logins (e.g. 5 in 15 min)	Low	High
CAPTCHA on register/login	reCAPTCHA / hCaptcha	Low	Medium

Fix	Description	Effort	Impact
Password complexity	Upper, lower, digit, symbol; min 10 chars	Low	Medium
Password breach check	HavelBeenPwned API k-anonymity	Low	Medium
Email verification	Verify email before first order	Medium	Medium
OAuth / social login	Google/Apple sign-in	High	Medium
Separate JWT secrets	Different secrets for admin vs mobile	Low	Medium
Short-lived access tokens	15–60 min access + refresh token	High	High
Refresh token rotation	One-time use refresh tokens	High	High
Server-side session store	Redis denylist for revoked JWTs	Medium	High
Logout API	Invalidate token server-side	Medium	Medium
Device binding	Track devices; alert on new login	High	Medium

8.2 Authorization & RBAC

Fix	Description	Effort	Impact
<code>requireRole('supplier')</code> middleware	Central role enforcement	Medium	High
Permission matrix document	Table of role × endpoint	Low	Medium
Sub-admin granular permissions	e.g. <code>canAdjustWallet</code> , <code>canEditTax</code> flags	Medium	High
Resource-level ACL	Order/supplier scoped by ownership	Medium	High
Admin action audit log	Who changed tax, wallet, verified supplier	Medium	High
Principle of least privilege	Review all admin routes	Medium	High
API scope tokens	Fine-grained scopes per client	High	Medium
Hide delivery partner PII	Mask phone until order assigned	Low	Low

8.3 API security

Fix	Description	Effort	Impact
<code>express-rate-limit</code>	Global + stricter on auth	Low	High
<code>express-slow-down</code>	Progressive delay on abuse	Low	Medium
Helmet	Security HTTP headers	Low	Medium
Request ID / correlation ID	Trace requests in logs	Low	Medium
JSON schema validation	zod/joi on all POST/PATCH bodies	Medium	High
API versioning	<code>/api/v1/</code> prefix	Medium	Low
IP allowlist for admin	Optional office IP restriction	Low	Medium
WAF (Cloudflare / AWS WAF)	Edge protection	Medium	High
Request signing for mobile	HMAC on sensitive requests	High	Medium
OpenAPI + security schemes	Document auth requirements	Low	Low

8.4 Input validation & injection prevention

Fix	Description	Effort	Impact
Escape RegExp special chars in search	<code>`search.replace(/[.*+?^\${}()]/g, '\\\\\$&')`</code>	Low	Medium
Parameterized queries only	Already using Mongoose — maintain discipline	Ongoing	High
Sanitize HTML in user content	Reviews, support messages	Medium	Medium
File upload validation	MIME, size, virus scan for supplier docs	Medium	Medium
Content Security Policy (admin)	CSP headers on admin static host	Low	Medium
CSRF tokens	If cookie auth added later	Medium	N/A today

8.5 Payment & wallet business logic

Fix	Description	Effort	Impact
Remove public wallet credit	Required — see R0-1	Low	Critical
Admin-only wallet top-up	Payment gateway → webhook → credit	Medium	High
Server-side price from DB	Required — see R1-1	Medium	Critical
Idempotent order creation	Idempotency-Key header	Medium	High
Razorpay webhook handler	<code>payment.captured</code> event	Medium	High
Amount + currency verify	Match order total in paise	Low	High
Store Razorpay order at create	Link create-order to verify	Low	High
Unique index <code>razorpayPaymentId</code>	MongoDB unique sparse index	Low	Medium
Timing-safe signature compare	<code>crypto.timingSafeEqual</code>	Low	Low
Fail closed on payment fetch error	Do not create order if fetch fails	Low	High

Fix	Description	Effort	Impact
Refund API integration	Razorpay refunds on cancel	Medium	Medium
Wallet ledger immutability	Append-only transactions	Medium	Medium
Daily reconciliation job	Compare Razorpay dashboard vs orders	Medium	High
PCI scope reduction	Never touch raw card data (Razorpay checkout only)	Ongoing	High

8.6 Mobile application security

Fix	Description	Effort	Impact
expo-secure-store for JWT	Encrypted storage	Low	High
Global auth guard in <code>_layout.tsx</code>	Redirect unauthenticated users	Medium	Medium
Certificate pinning	Pin API cert/public key	High	High
Root/jailbreak detection	Block or warn on compromised devices	Medium	Medium
ProGuard / R8 obfuscation	Android release builds	Low	Medium
Remove debug logs in production	Strip sensitive logs	Low	Medium
Deep link validation	Validate orderId format before API	Low	Low
Biometric unlock	Optional Face/Touch ID for app open	Medium	Medium
Auto-logout on background	Clear session after timeout	Low	Medium
Obfuscate API keys	Still extractable — rely on backend restrictions	Low	Low

8.7 Admin portal security

Fix	Description	Effort	Impact
httpOnly secure cookies	Alternative to localStorage (with CSRF care)	High	High
CSP + Subresource Integrity	Harden static assets	Medium	Medium
Sub-admin API restrictions	Mirror UI guards on all routes	Medium	High
Admin session timeout	Auto logout after inactivity	Low	Medium
2FA for admin users	TOTP mandatory for master/admin	High	High
IP logging for admin actions	Audit trail	Low	Medium

8.8 Network & infrastructure

Fix	Description	Effort	Impact
HTTPS with Let's Encrypt	Nginx reverse proxy	Medium	Critical
HSTS header	Force HTTPS	Low	High
TLS 1.2+ only	Disable old protocols	Low	Medium
Close MongoDB to VPC/IP allowlist	Atlas network access	Low	High
AWS Security Group least privilege	Only 443, 22 from known IPs	Low	High
Secrets Manager / Parameter Store	Not plain <code>.env</code> on disk	Medium	Medium
Separate staging environment	Never test exploits on prod	Medium	High
DDoS protection	Cloudflare or AWS Shield	Medium	Medium

8.9 Data security & privacy

Fix	Description	Effort	Impact
Field-level encryption	Phone, address at rest	High	Medium
PII retention policy	Auto-delete old addresses/logs	Medium	Medium
GDPR-style export/delete	User data export API	Medium	Medium
Log redaction	Never log passwords, tokens, PAN	Low	High
MongoDB backup encryption	Atlas backups	Low	Medium
Anonymize analytics	No PII in crash reports	Low	Medium

8.10 Monitoring, logging & incident response

Fix	Description	Effort	Impact
Centralized logging	CloudWatch, Datadog, ELK	Medium	High
Alert on failed admin logins	Spike detection	Low	High
Alert on wallet credit anomalies	Large credits	Low	High
Security incident runbook	Document response steps	Low	Medium
Regular dependency updates	Monthly npm audit	Ongoing	Medium
SAST in CI	Semgrep, CodeQL	Medium	Medium
DAST in CI	OWASP ZAP baseline scan	Medium	Medium
Bug bounty / external pen test	Before major launch	High	High

8.11 Compliance & governance

Fix	Description	Effort	Impact
PCI-DSS SAQ A	Razorpay hosted checkout — minimal scope	Low	Required for cards
Privacy policy alignment	Match actual data collected	Low	Legal
Terms of service	Payment, refunds, liability	Low	Legal
Security policy document	Internal team guidelines	Low	Governance
Vendor review	Razorpay, Atlas, AWS DPAs	Low	Compliance

9. Verification & Re-Test Checklist

After implementing fixes, re-run these checks.

9.1 Critical regression tests

- ☐ `POST /api/wallet/credit` returns **403 or 404** for normal users
- ☐ Default master admin credentials **do not work**
- ☐ Order with tampered `price: 1` is **rejected** (server uses DB price)
- ☐ `POST /api/orders` with `paymentMethod: "card"` returns **400** (must use Razorpay)
- ☐ API only accessible via **HTTPS** ; HTTP redirects or blocked
- ☐ Razorpay verify rejects amount mismatch

9.2 Authentication tests

- ☐ 20 failed logins trigger logout or CAPTCHA
- ☐ Password under minimum length rejected at register
- ☐ Logout invalidates token (if server denylist implemented)
- ☐ Expired JWT returns 401

9.3 Authorization tests

- ☐ Customer cannot access `/api/admin/*`
- ☐ Customer cannot access `/api/supplier/*`
- ☐ Sub-admin cannot access `/api/admin/financials`
- ☐ User A cannot fetch User B's order by ID

9.4 Admin UI tests

- ☐ Incognito `/orders` → login page
- ☐ Sub-admin `/financials` → blocked in UI and API

9.5 Mobile tests

- ☐ Logged-out user cannot complete checkout (API 401)
- ☐ Token stored in SecureStore (if implemented)
- ☐ Production build uses `https://` API URL

9.6 Dependency tests

```
cd backend && npm audit
cd admin && npm audit
cd mobile && npm audit
```

10. Compliance Notes (PCI-DSS & Privacy)

PCI-DSS

H2O uses **Razorpay Checkout** — card data is entered on Razorpay's UI, not stored in H2O MongoDB. This typically qualifies for **SAQ A** (card-not-present, outsourced).

Still required for PCI alignment:

- Use **HTTPS** for all payment-related pages and API calls
- Never log full card numbers or CVV
- Fix wallet/payment bypass flaws (Section 7) — integrity of payment flow

- Maintain secure admin access to payment reconciliation data

Data privacy (India / general)

- Store minimum PII (name, phone, address for delivery)
- Document retention in privacy policy
- Secure API keys and JWT secrets
- Allow users to request account deletion (future enhancement)

11. Appendix: Key Files Reference

Area	Path
API entry, CORS	backend/server.js
Mobile JWT auth	backend/middleware/auth.js
Admin JWT auth	backend/middleware/adminAuth.js
Login / register	backend/routes/auth.js
Admin login	backend/routes/adminAuth.js
Wallet credit (critical)	backend/routes/wallet.js
Orders create	backend/routes/orders.js
Order business logic	backend/services/customerOrderService.js
Razorpay	backend/routes/payments.js , backend/services/razorpay.js
User model / passwords	backend/models/User.js
Env validation	backend/config/env.js
Mobile token storage	mobile/src/context/AuthContext.js
Mobile API client	mobile/src/api/client.js
Build secrets	mobile/eas.json
Admin route guards	admin/src/App.jsx
Admin token	admin/src/api/client.js
Production env template	backend/.env.production.example

Document history

Version	Date	Author	Changes
1.0	July 2026	Security review (automated + manual)	Initial assessment and remediation guide

Next steps: Implement Section 7 fixes in priority order, then re-test with Section 9. For PDF export, add a generator script under docs/ (similar to generate-ba-guide-pdf.mjs) if needed.